

Aveo Compliance Desk — Plano de Execução

Hackathon: Colosseum / Superteam BR **Prazo oficial:** 12/10/2026, 23h59 PT (= 13/10, 03h59 em São Paulo) **Meta interna de envio:** 11/10/2026 **Base:** Aveo Protocol spec v3 (30/09/2026) **Equipe:** 4 pessoas, 4 frentes. O líder do projeto coordena todas as frentes e é dono das decisões de go/no-go.

Provas sintéticas; demo devnet; sem KYC real.

1. Visão em uma frase

O Aveo Compliance Desk responde: **esta carteira pode mover este ativo agora, qual prova sustenta a resposta e, se a prova vencer ou for revogada, quais ativos são afetados e como o operador fecha o incidente?**

2. O que a demo precisa provar

1. O verificador A emite uma atestação SAS sintética para a carteira X.
2. Dois tokens sintéticos, **Alfa e Beta**, com políticas independentes, aceitam a **mesma** prova de X sem nova emissão.
3. Transferências para X passam **também fora da interface** Aveo.
4. A carteira Y apresenta prova do verificador B: **Alfa aceita, Beta nega e explica o motivo.**
5. A prova de X expira ou é fechada: o Desk agrupa os dois ativos afetados e as transferências falham **mesmo com o Desk desligado.**
6. Uma prova nova é emitida, X assina a nova vinculação e o Desk registra a correção com evidência lida da blockchain.

Não prova: identidade real, conformidade jurídica, demanda de mercado ou integração com MPL-3643.

3. Escopo

Dentro

- Rota B: SAS real + Token-2022 + Transfer Hook próprio (Rust/Anchor), em validador local e devnet.
- Dois mints, dois verificadores (A e B), carteiras sintéticas X, Y e tesouraria.
- Desk em React com 3 telas: Carteiras/Ativos, Transferência, Incidentes.
- Incidentes em SQLite local (nunca usado como fonte de autorização).
- Script de transferência fora da UI.

Fora

KYC real, dados pessoais, ZK, bridge, DEX, token próprio, mainnet, SaaS multiusuário, monitor 24/7, auditoria, integração MPL-3643 (apenas citada como próximo passo).

Stretch (somente se tudo estiver verde após o freeze)

Copilot somente-leitura com Solana Agent Kit, atrás de `COPILLOT_ENABLED=false`, no máximo meio dia.

4. Stack

Camada	Escolha
Programa on-chain	Rust + Anchor 0.32.1
Toolchain	Solana CLI / Agave 2.3.x (confirmar no primeiro build)
Token	Token-2022 + Transfer Hook, sem PermanentDelegate e sem Confidential Transfer
Prova	Solana Attestation Service (SAS) real: credential, schema, attestation
Cliente	Node 22 + TypeScript, versões fixadas em lockfile
UI	React + TypeScript
Incidentes	SQLite local
Dev tooling	Solana AI Kit (revisado antes de instalar)

Regra: **nunca inventar program ID, offset ou versão de SDK**. Tudo que for usado é registrado em config/deployments.devnet.json e docs/sas-compatibility.md.

5. Arquitetura

```
Carteira —transfer—> Token-2022 —CPI—> aveo-hook (Execute)
                                     |
                                     |— lê (somente leitura)
                                     |— IssuerPolicy ["policy", mint]
                                     |— Binding origem ["binding", mint, owner_origem]
                                     |— Binding destino ["binding", mint, owner_destino]
                                     |— Contas SAS: credential, schema, attestation

Desk (React) —> backend-aveo (inspect / planTransfer / verifyOutcome)
                  ↳ scan:once —> SQLite (incidentes agrupados, sem duplicação)
```

O hook decide. O Desk explica, prepara e registra, mas **não concede permissão**.

6. Frentes de trabalho

O líder do projeto acompanha as 4 frentes, revisa os PRs e toma as decisões nos gates.

Frente	Responsabilidade	Pastas	Entrega principal
F1 — On-chain	IssuerPolicy, EligibilityBinding, Execute do hook, ExtraAccountMetaList, erros estáveis	programs/aveo-hook/	IDL, seeds e layouts; hook validando SAS real
F2 — SAS e infraestrutura	Credentials A/B, schemas, emissão/close de atestação, bootstrap dos mints, carteiras sintéticas, transferência fora da UI, testes on-chain	packages/sas-client/, scripts/, tests/onchain/, config/	Matriz T01–T17 automatizada
F3 — Backend do Desk	Contrato EligibilityBackend, adapter B, scan:once, incidentes em SQLite	packages/contracts/, packages/backend-aveo/, packages/backend-mpl/ (esqueleto), tests/client/	Diagnóstico, plano e verificação com readback
F4 — UI, docs e submissão	3 telas, estados unknown/erro, README, threat model, poderes, vídeo, pitch, formulário	app/, docs/	Demo gravada e submissão pronta

Arquivo compartilhado obrigatório no dia 1: docs/interface-v2.md (seeds, layouts, payload SAS, códigos de erro). Mudanças nele só por PR aprovado pelo líder.

Sugestão de perfil por frente

- F1 exige Rust/Anchor: alocar quem tem mais experiência em programação de sistemas.
- F2 e F3 são TypeScript com Solana: bons pontos de entrada para quem está aprendendo.
- F4 é o menor risco técnico e o maior impacto na nota de UX e apresentação.

7. Modelo de domínio

Conceito	Significado
Proof	Fato afirmado por um attestor sobre uma carteira (não é autorização universal)
Policy	Regras de um emissor para um mint
Eligibility	Resultado atual sob a policy e o relógio on-chain

Conceito	Significado
Binding	Ponteiro da carteira para a prova que ela apresenta naquele mint
Incident	Problema operacional confirmado (não é conclusão jurídica)
Evidence	Contas, slot, logs e resultado da operação

Políticas da demo

- **Alfa:** aceita verificador A ou B; exige KYC sintético.
- **Beta:** aceita apenas A; exige KYC + accreditation sintéticos.

Payload SAS aveo-eligibility-v2: version=2, subject_wallet[32], proof_domain[32], kyc_pass, accredited_pass. A validade vem de Attestation.expiry.

8. Regras que o hook valida

1. Está em contexto real de Transfer Hook (chamada direta falha).
2. Mint e policy corretas e ativas.
3. Bindings de origem e destino corretos (derivados do **owner** da token account, não do delegate).
4. Contas SAS com owner/PDA/layout oficiais e coerentes com o binding.
5. Par (credential, schema) permitido pela policy; signer ainda autorizado; schema não pausado.
6. Sujeito do payload = owner; fatos exigidos positivos; domínio correto.
7. expiry > Clock.unix_timestamp (expiry 0 não é aceito).
8. Qualquer conta ausente, fechada ou malformada: **nega**.

Erros estáveis: NotTransferContext, PolicyMissingOrInactive, MintMismatch, BindingMissing, BindingSubjectMismatch, ProviderPairNotAllowed, SchemaMismatchOrPaused, AttestationMissingOrClosed, InvalidSasOwnerOrData, UnauthorizedAttestationSigner, SubjectMismatch, ProofDomainMismatch, RequiredFactMissing, AttestationExpired, MissingExtraAccounts.

Erros do Desk: BackendNotIntegrated, UnsupportedCapability, ReadUnverifiable, SimulationFailed, ConfirmationUnknown.

9. Riscos principais

Risco	Impacto	Mitigação	Data-limite
Extra metas não conseguem resolver owner → binding → contas SAS	Perde o diferencial (bloqueio fora da UI)	Spike nos dias 1–2; alternativa: derivar a atestação direto das seeds (credential, schema, nonce) com nonce derivado da carteira + versão da prova	02/10
Estouro de compute (CU) ou tamanho de transação	Transferência falha sempre	Medir no spike; reduzir contas lidas	02/10
Incompatibilidade de versões Anchor/Agave/SAS	Build quebra	Fixar versões no dia 1 e documentar	01/10
Curva de aprendizado da equipe	Atraso geral	Estudo direcionado de meio dia; F1 com a pessoa mais experiente	01/10
Escopo crescendo	Não termina	Feature freeze em 08/10; stretch só com tudo verde	08/10
Envio no limite do prazo	Perder a submissão	Enviar em 11/10	11/10

10. Cronograma

Data	Entregas	Gate
30/09 – 01/10	Ambiente instalado e versões fixadas; interface-v2.md escrito; F2 emite e lê atestação SAS real; F1 inicia spike do hook com extra metas; F3/F4 montam esqueleto do monorepo e da UI	Atestação SAS lida por código

Data	Entregas	Gate
02/10	Gate P0: hook lê SAS real; extra metas resolvidas; close e CU medidos	Se falhar, adotar alternativa de seeds
03 – 04/10	Dois mints com a mesma prova; policies divergentes; transferência fora da UI permitida e negada (T01–T05); F3 com inspect e planTransfer reais	Go/no-go 04/10
05 – 06/10	Expiração/close, incidente agrupado, renovação + rebind assinado (T06–T09); UI ligada ao backend real	Fluxo completo local
07/10	Ponta a ponta em devnet; testes adversariais T10–T17	Falhas registradas, não escondidas
08/10	Feature freeze. README, lockfiles, threat model, poderes, tabela PASS/FAIL/NOT_RUN	Só correção de bug depois disso
09/10	Clone limpo em outra máquina; revisão humana de código, privacidade e licenças; copilot só se tudo verde (máx. meio dia)	Release candidate
10/10	Gravação da demo (~3 min) e pitch PT/EN	Vídeo aprovado pelo líder
11/10	Submissão	Enviado
12/10	Buffer de emergência	—

Pode ser cortado: copilot, adapter MPL, polimento visual. **Não pode ser cortado:** SAS real, bloqueio fora da UI, readback, testes adversariais, aviso de dados sintéticos.

11. Matriz de aceite

ID	Cenário	Resultado esperado
T01	X usa a mesma prova A em Alfa e Beta	Dois bindings, uma prova; ambos passam
T02	Y com prova B	Alfa passa; Beta nega par não aceito
T03	Prova A sem accreditation	Alfa passa; Beta nega requisito
T04	Binding/prova ausente ou KYC falso	Falha, saldos iguais
T05	Prova de X usada por Y; mint/domínio errado	Falha na validação
T06	Prova de X expira com Desk desligado	Entrada/saída de X falham nos dois ativos
T07	Close SAS autorizado	Próximas transferências falham; readback confirma
T08	A removido só de Alfa	Alfa bloqueia; Beta segue válido
T09	Renovação + rebind assinado por X	Prova nova validada; incidente resolvido
T10	Conta falsa / layout / schema / signer inválido	Falha fechada
T11	Fora do app, com e sem extras	Sem extras: falha, não bypass
T12	Delegate envia saldo de X	Hook usa owner X

ID	Cenário	Resultado esperado
T13	Update de policy/binding sem assinatura; Execute direto	Falha
T14	Timeout de RPC / leitura inconsistente	Unknown; incidente não fecha
T15	Scans repetidos / restart	Sem duplicação
T16	Mesmo owner nas duas contas / valores de fronteira	Sem desvio de sujeito
T17	Policy inativa / schema pausado / expiry 0 ou igual ao Clock	Falha

Aceite = teste on-chain real. UI verde ou mock não contam.

12. Estrutura do repositório

```

programs/aveo-hook/      F1
packages/contracts/      F3 (interface compartilhada)
packages/sas-client/      F2
packages/backend-aveo/    F3
packages/backend-mpl/     F3 (esqueleto "not-integrated")
app/                      F4
scripts/                  F2 (bootstrap, scan:once, outside-ui-transfer)
tests/onchain/            F2 + F1
tests/client/             F3
docs/                     F4 (interface-v2, threat-model, demo, provenance)
config/                   F2 (manifests, sem secrets)

```

Regras de trabalho: uma branch por frente, PR revisado pelo líder, chaves e .env nunca no Git.

13. Plano de aprendizado (antes de codar, ~4–6 h)

Tópico	Quem	Material
Modelo de contas, PDAs, seeds	Todos	https://solana.com/docs/core
Token-2022 Transfer Hook	Todos	https://solana.com/docs/tokens/extensions/transfer-hook
Integração do hook no cliente (extra metas)	F1, F2, F3	https://solana.com/docs/tokens/extensions/transfer-hook-integration
SAS: attestations, credentials, schemas, close	F2, F3	https://attest.solana.com/docs/attestations
Anchor basics e PDAs	F1	https://www.anchor-lang.com/docs
Prática: CLI, carteira devnet, airdrop, transferência em TS	Todos	Solana CLI + @solana/kit

Recomendação: não estudar Rust de forma geral. Aprender fazendo o spike, com IA como apoio, e sempre validar por build e teste.

14. Demo e pitch

Roteiro (~3 min): políticas Alfa/Beta → prova de X compartilhada → transferência fora da UI → Y passa em Alfa e falha em Beta → close da prova de X → scan agrupa incidente → script externo falha com Desk parado → renovação + rebind → transferência permitida e incidente resolvido com evidência.

Pitch (EN):

We started with portable eligibility for assets on Solana. During our research, we found Metaplex's MPL-3643, which already covers identity, issuer policies and permissioned tokens. We are not claiming to have invented those building blocks. Aveo is a small compliance desk: one synthetic attestation can support two assets, each issuer keeps its own policy, and an operator can trace an eligibility incident through to a verified fix. Our demo uses SAS and Token-2022 and shows

on-chain enforcement outside our UI. This submission does not integrate MPL-3643; an official adapter is future work, subject to access and testing.

15. Checklist de submissão

- ☐ README reproduzível a partir de clone limpo
- ☐ Lockfiles e versões registradas
- ☐ Program IDs, mints e contas em `config/deployments.devnet.json`
- ☐ Tabela PASS / FAIL / NOT_RUN da matriz T01–T17
- ☐ Transações confirmadas e logs das recusas (sem links de explorer inventados)
- ☐ Threat model e poderes residuais (issuer, upgrade authority, hook authority, mint/burn)
- ☐ Vídeo da demo
- ☐ Pitch e plano de negócio no formulário
- ☐ Verificação de elegibilidade de cada participante nas regras do Colosseum
- ☐ Revisão de privacidade: nenhuma carteira pessoal, CPF ou dado real

16. Rituais do líder

- **Daily de 15 min:** o que foi feito, o que bloqueia, gate do dia.
- **Gates com decisão registrada:** 02/10 (P0), 04/10 (go/no-go), 08/10 (freeze), 10/10 (vídeo).
- **Revisão de PR:** nenhuma mudança em `interface-v2.md` ou no programa sem aprovação.
- **Registro de bloqueios:** qualquer falha vira item documentado, nunca é escondida por mock.